Astro 142 Discussion 17

2025-11-25

Dyson Lewis

Discussion Activity

Discussion 17 Activity

There are two activity templates.

1. Modify the script to work with GNU parallel, and compare the runtimes for each technique.
2. Modify the script to parse arguments from the command line, calculate the trajectory and save data and plots.

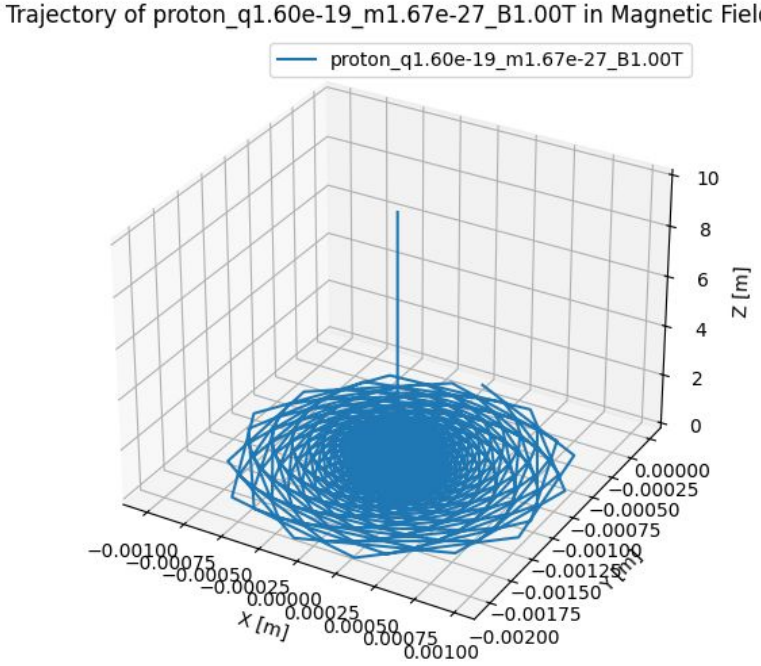
When you are done, put screenshots in the following slides and submit a pdf.

```python
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import median_abs_deviation
from datetime import datetime
import logging
import sys

def process_order(ordernumber, basename='/home/dyson/fall25/142ASTR/week8/Disc15/
    """Process a single order and save the result."""
    data = np.load(basename+str(ordernumber)+'.npy')
    logging.info('Loaded file: ' + basename+str(ordernumber)+'.npy')

    if plot:
        plt.imshow(data, aspect=15, vmin=0, vmax=1e4)
        plt.show()

    ### Scale each exposure to a consistent median
    for i in range(data.shape[0]):
        data[i] = data[i]/np.nanmedian(data[i])

    ### Now we want to make the median over the time series (y-axis)
    medspec = np.nanmedian(data, axis=0)

    ### Now divide each spectrum in the time series by its median
    for i in range(data.shape[0]):
        data[i] = data[i]/medspec

    ### And let's mask outliers. Use a boolean index for speed
    threshold = 6*median_abs_deviation(data.flatten())
    data[np.abs(data-1)>threshold] = np.nan

    if plot:
        plt.imshow(data, aspect=15, vmin=0.95, vmax=1.05)
        plt.show()

    logging.info('Done with ' + basename+str(ordernumber)+'.npy')
    return data

if __name__ == '__main__':
    # Check if order number is provided as command line argument
    if len(sys.argv) > 1:
        # Single order mode - for use with GNU parallel
```

```python
        # Single order mode - for use with GNU parallel
        order = int(sys.argv[1])
        logging.basicConfig(
            filename=f'logfile_order_{order}.log',
            level=logging.INFO
        )

        t_start = datetime.now()
        logging.info(f'Starting order {order}')

        data = process_order(order)

        # Save the output
        output_file = f'WASP33_nov21_processed_order{order}.npy'
        np.save(output_file, data)
        logging.info(f'Saved {output_file}')

        t_end = datetime.now()
        logging.info(f'Order {order} time: {t_end - t_start}')

    else:
        # Run all orders sequentially for comparison
        logging.basicConfig(filename='logfile.log', level=logging.INFO)
        orders = [0, 1, 2, 3, 4, 5, 6, 7, 8]

        t1 = datetime.now()
        logging.info('Starting sequential processing')
        for order in orders:
            data = process_order(order)
            np.save(f'WASP33_nov21_processed_order{order}.npy', data)
        t2 = datetime.now()
        logging.info(f'Sequential
```

```
dyson@dpl-linux:~/fall25/142ASTR/week9/disc16$ time parallel -j 9 python3 discus
sion17_activity1_template.py ::: {0..8}
/home/dyson/fall25/142ASTR/week9/disc16/discussion17_activity1_template.py:22: R
untimeWarning: All-NaN slice encountered
  medspec = np.nanmedian(data, axis=0)
/home/dyson/fall25/142ASTR/week9/disc16/discussion17_activity1_template.py:22: R
untimeWarning: All-NaN slice encountered
  medspec = np.nanmedian(data, axis=0)
/home/dyson/fall25/142ASTR/week9/disc16/discussion17_activity1_template.py:22: R
untimeWarning: All-NaN slice encountered
  medspec = np.nanmedian(data, axis=0)
/home/dyson/fall25/142ASTR/week9/disc16/discussion17_activity1_template.py:22: R
untimeWarning: All-NaN slice encountered
  medspec = np.nanmedian(data, axis=0)
/home/dyson/fall25/142ASTR/week9/disc16/discussion17_activity1_template.py:22: R
untimeWarning: All-NaN slice encountered
  medspec = np.nanmedian(data, axis=0)

real    0m1.395s
user    0m13.241s
sys     0m3.061s
```

```python
import argparse
import numpy as np
import matplotlib.pyplot as plt

### We are further expanding on the particle trajectory example from week 2.
### Modify the script below to allow command line inputs
### to set the initial parameters of the simulation, such as particle type, charge, mass, initial velocity components,
### time step, and number of steps.

# ---- Argument Parser Setup ----
parser = argparse.ArgumentParser(description="Calculate and plot the trajectory of charged particles in a magnetic field.
parser.add_argument("-particle", type=str, default="proton", help="Type of particle: proton, alpha, deuteron")
parser.add_argument("-q", type=float, default=1.6e-19, help="charge of the particle (Coulombs)")
parser.add_argument("-m", type=float, default=1.67e-27, help="mass of the particle (kg)")
parser.add_argument("-vx", type=float, default=1e5, help="initial velocity in x direction (m/s)")
parser.add_argument("-vy", type=float, default=0.0, help="initial velocity in y direction (m/s)")
parser.add_argument("-vz", type=float, default=1e5, help="initial velocity in z direction (m/s)")
parser.add_argument("-x0", type=float, default=0.0, help="initial x position (m)")
parser.add_argument("-y0", type=float, default=0.0, help="initial y position (m)")
parser.add_argument("-z0", type=float, default=0.0, help="initial z position (m)")
parser.add_argument("-Bx", type=float, default=0.0, help="magnetic field in x direction (Tesla)")
parser.add_argument("-By", type=float, default=0.0, help="magnetic field in y direction (Tesla)")
parser.add_argument("-Bz", type=float, default=1.0, help="magnetic field in z direction (Tesla)")
parser.add_argument("-dt", type=float, default=1e-8, help="time step (seconds)")
parser.add_argument("-steps", type=int, default=10000, help="number of time steps")
parser.add_argument("-plot", action="store_true", help="display plot interactively")
parser.add_argument("-nosave", action="store_true", help="don't save plot to file")

# ---- Parse Arguments ----
args = parser.parse_args()

# ---- Function Definition ----

def acceleration(q, m, v, B_field):
    return (q/m) * np.cross(v, B_field)

# use RK4 numerical integration to calculate position at each time step
def rk4_integration(q, m, r0, v0, B_field, dt, steps):
    r = np.zeros((steps, 3))
    v = np.zeros((steps, 3))
    r[0], v[0] = r0, v0

def plot_trajectory(r, label, save=True, plot=False):
    fig = plt.figure(figsize=(8, 6))
    ax = fig.add_subplot(111, projection='3d')
    ax.plot(r[:,0], r[:,1], r[:,2], label=label)
    ax.set_title(f'Trajectory of {label} in Magnetic Field')
    ax.set_xlabel('X [m]')
    ax.set_ylabel('Y [m]')
    ax.set_zlabel('Z [m]')
    ax.legend()
    if save:
        plt.savefig(f'{label}_trajectory.png')
        print(f"Saved plot to {label}_trajectory.png")
    if plot:
        plt.show()

# ---- Main Execution ----

if __name__ == "__main__":
    # Update particle parameters based on command line arguments
    particle_types = {
        "proton": {"q": 1.6e-19, "m": 1.67e-27},
        "alpha": {"q": 2*1.6e-19, "m": 4*1.67e-27},
        "deuteron": {"q": 1.6e-19, "m": 2*1.67e-27}
    }

    if args.particle in particle_types:
        q = particle_types[args.particle]["q"]
        m = particle_types[args.particle]["m"]
        print(f"Using predefined {args.particle} parameters: q={q:.2e} C, m={m:.2e} kg")
    else:
        print(f"Using custom particle parameters: q={args.q:.2e} C, m={args.m:.2e} kg")
        q = args.q
        m = args.m

    # Set other parameters from args
    r0 = np.array([args.x0, args.y0, args.z0])
    v0 = np.array([args.vx, args.vy, args.vz])
    B_field = np.array([args.Bx, args.By, args.Bz])
    dt = args.dt
    steps = args.steps

    # Print simulation parameters
    print(f"Initial position: {r0}")
```


Trajectory of proton_q1.60e-19_m1.67e-27_B1.00T in Magnetic Field
proton_q1.60e-19_m1.67e-27_B1.00T
Z [m]
10
8
6
4
2
0
X [m]
−0.00100
−0.00075
−0.00050
−0.00025
0.00000
0.00025
0.00050
0.00075
0.00100
Y [m]
0.00000
−0.00025
−0.00050
−0.00075
−0.00100
−0.00125
−0.00150
−0.00175
−0.00200